



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Уральский федеральный университет имени первого Президента России Б.Н. Ельцина» (УрФУ)
Институт радиоэлектроники и информационных технологий – РтФ
Школа бакалавриата

Отчет

По дисциплине «Цифровые двойники»

Студент: Иноземцев Кирилл Евгеньевич _____
ФИО подпись

Группа: РИЗ-151105у _____

Екатеринбург
2026

КОНСПЕКТ ЛЕКЦИИ

«Возможные проекты на Unity & C#. Обзор редактора»

Что такое Unity

Unity — кроссплатформенный игровой движок для создания интерактивного контента в реальном времени. Поддерживает платформы ПК, консоли, мобильные устройства, VR/AR-гарнитуры. Сценарии (скрипты) в Unity создаются двумя способами: программным — на языке C# (Programming Language), и визуальным — с помощью системы Bolt (visual programming).

Возможности Unity

Unity позволяет создавать широкий спектр продуктов:

- простые игры с минимальным взаимодействием, графикой и сложностью;
- сложные игры для консолей, ПК и мобильных устройств (примеры: Ori and the Blind Forest, Firewatch);
- интерактивные пространства и обучающие симуляторы;
- VR-симуляторы: в медицине (Project VR med), в инженерии (NovaTrans), Ansys VR Experience;
- виртуальные лабораторные стенды для образования.

Требования к разработчику (формула 50/50)

Для комфортной работы в Unity необходимо сочетать две области знаний в равной мере:

- **Game Design:** подходы к разработке игр; основы игрового дизайна — Механика, Динамика, Эстетика (Тетрада); дизайн окружения; управление игроком; игровой баланс.
- **Разработка:** C#, ООП, Git, JavaScript.

Unity + AI и образовательные применения

Unity используется не только для игр. Ключевые направления: обучающие симуляторы, приложения экстренного реагирования (first-responder applications), бизнес-приложения с 2D/3D-пространством. Для AI-задач применяется пакет **Unity ML-Agent**, позволяющий обучать агентов методами машинного обучения прямо в сцене.

В образовании Unity применяется для: создания интерактивных лабораторных работ, разработки виртуальных стендов (пример: виртуальный стенд

«Вытеснение металла из раствора соли»), проведения виртуальных экспериментов. В дипломных проектах и магистерских работах Unity используется для построения визуальных моделей разной сложности.

Обзор интерфейса Unity

Установка: **unity.com** → «Get started» → Unity Hub. Для создания проекта задаются имя, расположение и шаблон (2D, 3D, URP, HDRP и др.).

Структура проекта включает папки: Assets (все ресурсы проекта), ProjectSettings, Library. Папки Library и ProjectSettings не редактируются вручную.

Основные панели редактора:

- **Scene** — окно редактирования сцены;
- **Game** — предпросмотр финального вида;
- **Hierarchy** — иерархия объектов сцены;
- **Inspector** — свойства выбранного объекта;
- **Project** — файловая структура Assets;
- **Console** — вывод ошибок и отладочных сообщений.

Связь лекционного материала с практикой

Лекция охватила широкий спектр применений Unity — от игровой разработки до VR-симуляторов и образовательных проектов. Разработанная в рамках лабораторной работы симуляция гравитационного взаимодействия планеты и астероида является примером именно образовательного проекта — цифрового двойника физического явления, недоступного для прямого наблюдения.

Проект соответствует разделу «Образовательные проекты на Unity»: симуляция выступает в роли виртуального эксперимента, наглядно демонстрирующего законы небесной механики — траекторию падения тела в поле тяготения, атмосферное торможение и удар о поверхность.

Что было реализовано

В ходе выполнения работы в Unity была разработана сцена с двумя физическими телами: планетой (масса 10 000 у.е.) и астероидом (масса 10 у.е., начальная скорость 60 ед/с). Симуляция демонстрирует следующие явления:

- гравитационное притяжение согласно закону Ньютона, реализованное в скрипте GravityBody.cs;
- атмосферное торможение при прохождении астероида через триггерную зону Atmosphere с эффектом горения (ParticleSystem);
- траектория полёта с разворотом под действием гравитации (TrailRenderer);
- образование кратера, лавы и дымовых облаков при ударе о поверхность (AsteroidEffects.cs).

Применённые инструменты Unity из лекции

В процессе работы были использованы все ключевые элементы, упомянутые в лекционном материале: редактирование в панелях Scene и Hierarchy, настройка компонентов через Inspector, создание и управление ресурсами через папку Assets, написание скриптов на C# с использованием принципов ООП, работа с версиями через Git (репозиторий на GitHub).

Трудности и выводы

Основная сложность состояла в подборе начальных условий симуляции: баланс между гравитационной константой ($G = 2$) и начальной скоростью астероида ($v_0 = 60$) определяет характер траектории. При слишком малой скорости

астероид немедленно падает; при слишком большой — покидает сцену. Это хорошо иллюстрирует принцип «игрового баланса», о котором говорилось в разделе Game Design.

Формула «50/50», предложенная на лекции, подтвердилась на практике: техническая реализация физики потребовала знания C# и ООП, тогда как визуальная наглядность и понятность сценария — навыков дизайна сцены и работы с эффектами. Проект продемонстрировал, что Unity является полноценным инструментом для создания образовательных цифровых двойников.

ОПИСАНИЕ ПРОЕКТА

Постановка задачи

Разработана симуляция физического явления — падения астероида на планету с атмосферой. Проект реализован в Unity (URP) и является образовательным цифровым двойником астрофизического процесса. Сценарий: астероид вылетает из заданной точки, проходит через атмосферу (сгорая), разворачивается под действием гравитации и падает на поверхность планеты с образованием кратера.

3.2 Объекты сцены

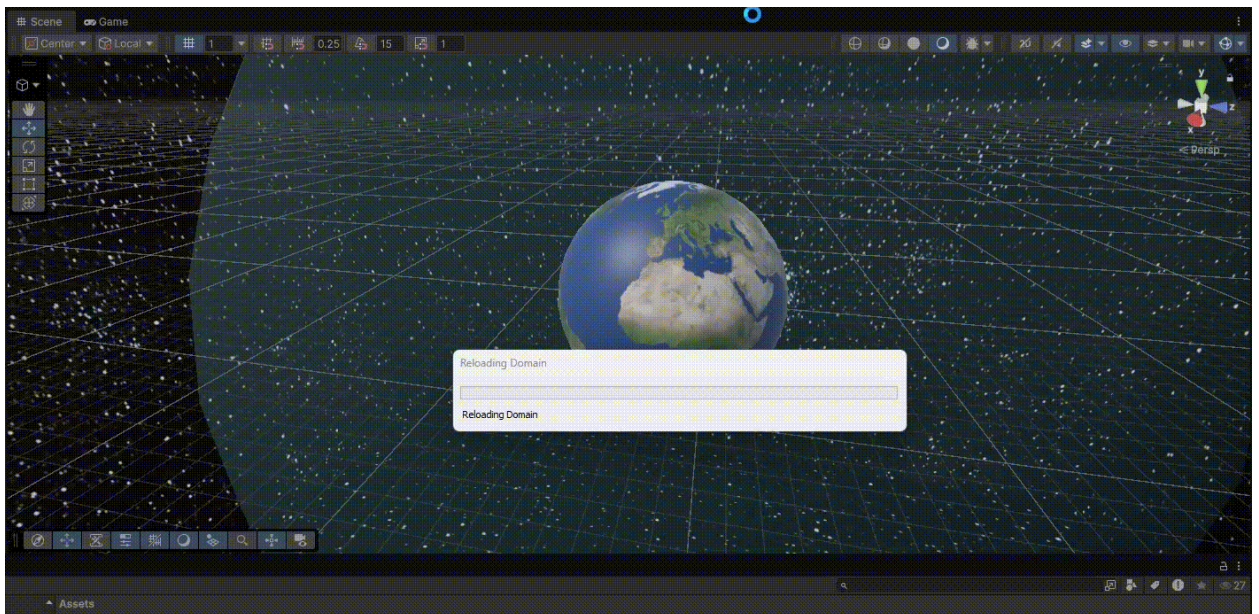
Объект	Параметры	Описание
Planet	Mass: 10 000, G: 2	Планета; вращается (speed=5); с атмосферой и коллайдером
Asteroid	Mass: 10, $v_0=(0,0,60)$	Астероид с TrailRenderer, эффектами горения и кратера
Atmosphere	Trigger-зона	Сфера вокруг планеты; при входе — активация BurnEffect + торможение
BurnEffect	ParticleSystem	Частицы горения при прохождении через атмосферу
MagneticZone	Tag: MagneticField	Зона магнитного поля для MagneticSensor
Particle System	ParticleSystem	Дым и обломки при ударе о поверхность

Пользовательские скрипты (C#)

- **GravityBody.cs** — вычисление $F = G \cdot m_1 \cdot m_2 / r^2$ и применение через `Rigidbody.AddForce()` в `FixedUpdate`;
- **AsteroidEffects.cs** — управление `burnParticles` и создание `craterPrefab` при столкновении;
- **PlanetRotation.cs** — вращение планеты (`rotationSpeed = 5`) в `Update`;
- **MagneticSensor.cs** — детектирование `MagneticZone`, активация `sensorParticles`.

Алгоритм работы

- Старт: астероид получает начальную скорость $(0, 0, 60)$ — по касательной к планете.
- GravityBody пересчитывает вектор силы каждый физический кадр; Rigidbody движется по криволинейной траектории.
- Вход в Atmosphere (триггер): активируется BurnEffect, к скорости добавляется тормозящая компонента.
- Под действием гравитации астероид разворачивается и повторно входит в атмосферу.
- Столкновение с Planet: AsteroidEffects порождает craterPrefab (кратер + лава) и запускает дымовые частицы. Прикладываю видео процесс как это выглядит:



ССЫЛКА НА РЕПОЗИТОРИЙ GITHUB

Проект размещён в открытом репозитории на платформе GitHub и содержит сцену Unity, пользовательские C#-скрипты, материалы сцены и файл README.md с полным описанием проекта.

Репозиторий: <https://github.com/scaringeboosalis-eng/SimulatorEarthArmagedonInozemtsevv>

В репозитории оформлен файл README.md, содержащий описание проекта, перечень реализованных физических явлений, технические параметры, описание скриптов и инструкцию по запуску.